

Estrutura do app

Estrutura do app

Este documento explica como o app está estruturado e onde estão feitas as chamadas de componentes JSX, rotas, models, controllers e como o fluxo Inertia/Vite/Blade funciona.

Arquitetura geral

- Rotas (backend): `routes/*.php` — definem endpoints HTTP e mapeiam para controllers ou diretamente para páginas Inertia com `Route::inertia()`.
- Controllers: `app/Http/Controllers/*` — lógica de backend, validação, chamadas a modelos e retorno de respostas. Para páginas Inertia, usam `Inertia::render('component/name', $props)`.
- Models: `app/Models/*` — Eloquent models que representam tabelas do banco.
- Views Blade: `resources/views/*.blade.php` — arquivo principal `app.blade.php` monta o ponto de entrada do Inertia (container onde o app React é hidratado).
- Frontend (JSX/TSX): `resources/js/pages/*` (páginas), `resources/js/components/*` (componentes), `resources/js/layouts/*` (layouts reutilizáveis), `resources/js/app.tsx` (bootstrap do Inertia + configuração de layout/hmr/wrappers).
- Assets/Build: Vite + `@vite` no Blade injeta os bundles gerados; `@viteReactRefresh` habilita HMR para React em tempo de desenvolvimento.

Fluxo de uma requisição que renderiza uma página

1. Navegador acessa uma URL definida em `routes/web.php`.
2. A rota pode ser:
 - `Route::inertia('path', 'component/name')` — macro do pacote `inertia-laravel` que responde com dados Inertia e indica o componente frontend a renderizar.
 - Ou `Route::get/post(..., [Controller::class, 'method'])` — o controller pode chamar `Inertia::render('component/name', $props)`.
3. O Inertia responde com um payload JSON contendo `component` (ex.: `settings/profile`) e `props`.
4. No frontend, `resources/js/app.tsx` (via `createInertiaApp`) recebe esse `component` e carrega o arquivo em `resources/js/pages/{component}.tsx`.
5. O componente React renderiza, usando `layout` definido no próprio componente (ex.: `Dashboard.layout = {...}`) ou a função `layout` definida em `app.tsx` para envolver o componente em `AppLayout`, `AuthLayout`, etc.
6. `resources/views/app.blade.php` contém o componente Blade `<x-inertia::app />` que monta o container e injeta os scripts/styles (via `@vite`) para hidratar o app React.

Onde ficam as chamadas JSX / páginas

- `resources/js/pages/` — cada arquivo exporta o componente que representa uma rota Inertia. Exemplos:

- `resources/js/pages/dashboard.tsx` — componente `Dashboard`.
- `resources/js/pages/settings/*` — páginas de configuração.
- `resources/js/app.tsx` — configura `createInertiaApp`:
- `title` — formata o título da página.
- `layout` — função que escolhe qual layout aplicar com base no nome do componente.
- `withApp` — wrapper comum que permite injetar providers (`Toaster`, `TooltipProvider`).
- `progress` — configura barra de progresso de navegação.

Rotas e controllers

- `routes/web.php` — define rotas públicas e usa `Route::inertia()` para páginas simples (ex.:

``Route::inertia('/', 'welcome')``).

- ``routes/settings.php`` — mostra exemplos onde rotas usam controllers para métodos ``edit``, ``update``, ``destroy`` e também ``Route::inertia`` para páginas que não precisam de controller.
- Controllers podem retornar ``Inertia::render('settings/profile', $props)`` — isso passa dados ao frontend via ``props``.

Models

- ``app/Models`` — modelos Eloquent (ex.: ``User.php``). São usados pelos controllers para consultar, criar e atualizar dados.
- Migrations em ``database/migrations`` definem o esquema do banco.

Blade e Vite

- ``resources/views/app.blade.php`` — arquivo Blade principal. Responsabilidades:
 - Definir HTML base e metatags.
 - Aplicar tema (``dark``) precoce com script inline e atributo ``@class(...)`` para evitar flash de tema.
 - Incluir favicons e fontes.
 - Inserir diretivas ``@vite`` e ``@viteReactRefresh`` para carregar assets gerados pelo Vite.
 - Usar componentes Blade do Inertia: `<x-inertia::head>` e `<x-inertia::app />`.

Como os dados chegam ao JSX

- Quando um controller chama ``Inertia::render('component', ['foo' => 'bar'])``, o objeto ``props`` chega ao componente React como props (ou via hook ``usePage()`` se preferir).
- O Inertia controla navegação SPA usando XHR/fetch; quando a navegação é feita via link Inertia, apenas ``props`` novos são carregados.

Alias e import paths

- O projeto usa alias ``@/...`` em imports (ex.: ``@/components/...``). Esses aliases são resolvidos pelo ``vite.config.ts`` e ``tsconfig.json``.

Exemplo rápido (resumido)

- Rota: ``Route::inertia('dashboard', 'dashboard')``
- Arquivo frontend: ``resources/js/pages/dashboard.tsx``
- Layout aplicado: ``AppLayout`` a partir de ``resources/js/layouts/app-layout.tsx``
- Arquivo Blade: ``resources/views/app.blade.php`` monta o container Inertia e inclui os assets via Vite.

(Documento gerado automaticamente pelo assistente.)